

LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale

LLM.int8(): 대규모 Transformer를 위한 8비트 행렬 곱셈

원저자: Tim Dettmers (University of Washington), Mike Lewis (Facebook AI Research), Younes Belkada (Hugging Face, ENS Paris-Saclay), Luke Zettlemoyer (University of Washington, Facebook AI Research)
원문: arXiv:2208.07339v2 [cs.LG] (2022-11-10, Version 2) · NeurIPS 2022 · 코드: <https://github.com/TimDettmers/bitsandbytes>

본 문서는 학습 목적의 한국어 번역본이다. 개념을 나타내는 전문 용어는 「한국어(영어)」 형식으로 병기했으며, 약어(FFN, PPL 등)는 최초 등장 시 풀어쓰고 이후 약어로 표기한다. 고유명사, 모델명·데이터셋명, 그림·표 라벨, 수식 기호는 영어 원문 그대로 둔다. 그림은 원문 PDF에서 잘라내어 삽입했고, 표는 원문 수치를 그대로 다시 조판했으며, 캡션 번역을 파란 상자로 덧붙였다. 참고문헌(References)과 NeurIPS 체크리스트는 고유명사·서지 정보의 나열이므로 원문을 참조한다.

목차

- 초록 (Abstract)
- 1. 서론 (Introduction)
- 2. 배경 (Background)
- 3. 대규모에서의 Int8 행렬 곱셈 (Int8 Matrix Multiplication at Scale)
- 4. 대규모 Transformer에서 창발하는 큰 크기의 특성 (Emergent Large Magnitude Features in Transformers at Scale)
- 5. 관련 연구 (Related Work)
- 6. 논의와 한계 (Discussion and Limitations)
- 7. 더 넓은 영향 (Broader Impacts)
- 부록 A-F (Appendix)

초록 (Abstract)

대형 언어 모델은 널리 채택되었지만 추론(inference)에 상당한 GPU 메모리를 요구한다. 우리는 Transformer의 피드포워드(feed-forward) 계층과 어텐션 투영(attention projection) 계층을 위한 Int8 행렬 곱셈 절차를 개발한다. 이 절차는 완전한 정밀도의 성능을 유지하면서 추론에 필요한 메모리를 절반으로 줄인다. 우리의 방법을 쓰면 175B 파라미터의 16/32비트 체크포인트를 불러와 Int8로 변환한 뒤 성능 저하 없이 즉시 사용할 수 있다. 이는 어텐션과 Transformer의 예측 성능을 지배하는, 매우 체계적인 창발적 특성(emergent feature)의 속성을 이해하고 우회함으로써 가능해진다. 이 특성들에 대처하기 위해 우리는 2단계 양자화(quantization) 절차인 LLM.int8()을 개발한다. 먼저 행렬 곱셈 안의 각 내적(inner product)마다 별도의 정규화 상수를 두는 벡터 단위 양자화(vector-wise quantization)로 대부분의 특성을 양자화한다. 그러나 창발적 이상치(outlier)에 대해서는, 이상치 특성 차원을 16비트 행렬 곱셈으로 분리하되 여전히 99.9% 이상의 값을 8비트로 공급하는 새로운 혼합 정밀도 분해(mixed-precision decomposition) 방식을 추가로 도입한다. LLM.int8()을 사용하면 최대 175B 파라미터의 LLM에서 어떤 성능 저하도 없이 추론을 수행할 수 있음을 실증적으로 보인다. 이 결과는 그런 모델들의 접근성을 크게 높여, 예컨대 OPT-175B/BLOOM을 소비자용 GPU를 갖춘 단일 서버에서 사용할 수 있게 한다. 우리는 소프트웨어를 오픈 소스로 공개한다.

1. 서론 (Introduction)

대형 사전학습(pre-training) 언어 모델은 NLP에서 널리 채택되었지만 추론에 상당한 메모리를 요구한다. 6.7B 파라미터 이상의 대형 Transformer 언어 모델에서는 피드포워드 계층과 어텐션 투영 계층, 그리고 그 행렬 곱셈 연산이 소비되는 파라미터의 95%²와 전체 계산의 65-85%를 차지한다. 파라미터 크기를 줄이는 한 방법은 더 적은 비트로 양자화하여 저비트 정밀도 행렬 곱셈을 쓰는 것이다. 이 목표로 Transformer를 위한 8비트 양자화 방법들이 개발되어 왔다. 이 방법들은 메모리 사용을 줄이지만 성능을 저하시키고, 보통 학습 후 추가적인 양자화 튜닝을 요구하며, 350M 파라미터 미만의 모델에서만 연구되었다. 350M 파라미터까지의 무손실 양자화도 잘 이해되지 않은 상태이며, 수십억 파라미터 규모의 양자화는 여전히 열린 난제다.

이 논문에서 우리는 어떤 성능 저하도 일으키지 않는 최초의 수십억 규모(multi-billion-scale) Transformer용 Int8 양자화 절차를 제시한다. 이 절차를 쓰면 16비트 또는 32비트 가중치를 가진 175B 파라미터 Transformer를 불러와 피드포워드 계층과 어텐션 투영 계층을 8비트로 변환한 뒤, 결과 모델을 성능 저하 없이 즉시 추론에 사용할 수 있다. 우리는 두 가지 핵심 난제를 해결하여 이 결과를 달성한다: 1B 파라미터를 넘는 규모에서 더 높은 양자화 정밀도가 필요하다는 점, 그리고 6.7B 파라미터 규모부터 모든 Transformer 계층에서 나타나 양자화 정밀도를 망가뜨리는, 희소하지만 체계적인 큰 크기의 이상치 특성(outlier feature)을 명시적으로 표현해야 한다는 점이다. 이 정밀도 손실은 이 이상치 특성이 창발하는 즉시 C4 평가 퍼플렉시티(perplexity, 3절)와 제로샷(zeroshot) 정확도에 반영되며, Figure 1에서 볼 수 있다.

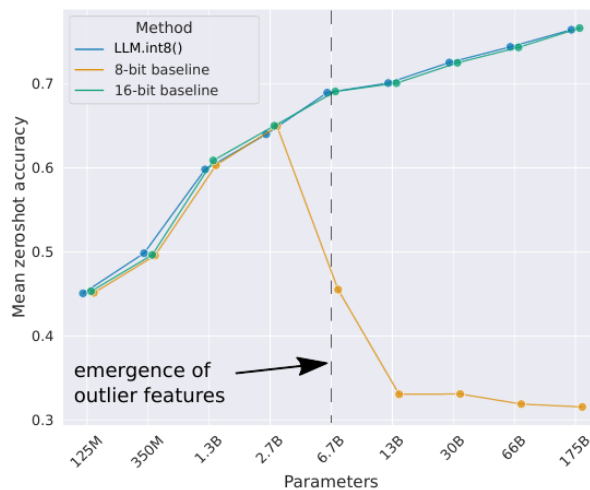


Figure 1: WinoGrande, HellaSwag, PIQA, LAMBADA 데이터셋에 대한 OPT 모델의 평균 제로샷 정확도. 16비트 기준선, 가장 정밀한 기존 8비트 양자화 방법을 기준선으로, 그리고 우리의 새 8비트 양자화 방법 LLM.int8()을 나타냈다. 6.7B 파라미터 규모에서 체계적 이상치가 나타나는 순간 통상적인 양자화 방법은 실패하는 반면, LLM.int8()은 16비트 정확도를 유지함을 볼 수 있다.

우리 방법의 첫 부분인 **벡터 단위 양자화(vector-wise quantization)**로는 2.7B 파라미터 규모까지 성능을 유지할 수 있음을 보인다. 벡터 단위 양자화에서는 행렬 곱셈을 행 벡터와 열 벡터의 독립적인 내적들의 나열로 볼 수 있다. 따라서 내적마다 별도의 양자화 정규화 상수를 사용하여 양자화 정밀도를 높일 수 있다. 다음 연산을 수행하기 전에 열·행 정규화 상수의 외적(outer product)으로 역정규화(denormalize)하면 행렬 곱셈의 출력을 복원할 수 있다.

성능 저하 없이 6.7B 파라미터 너머로 확장하려면, 추론 중 은닉 상태(hidden state)의 특성 차원에서 극단적 이상치가 창발하는 현상을 이해하는 것이 결정적이다. 이를 위해 우리는 새로운 기술적(descriptive) 분석을 제공한다.

이 분석은 다른 차원보다 크기가 최대 20배 큰 특성이 처음에는 전체 Transformer 계층의 약 25%에서 나타나다가, Transformer를 6B 파라미터로 키우면서 점차 다른 계층으로 퍼져 나감을 보여준다. 약 6.7B 파라미터에서 상전이(phase shift)가 일어나 모든 Transformer 계층과 전체 시퀀스 차원의 75%가 극단적 크기의 특성에 영향을 받는다. 이 이상치들은 매우 체계적이다: 6.7B 규모에서 시퀀스당 150,000개의 이상치가 발생하지만, 이들은 Transformer 전체에서 단 6개의 특성 차원에 집중되어 있다. 이 이상치 특성 차원을 0으로 만들면, 전체 입력 특성의 약 0.1%에 불과함에도 top-1 어텐션 소프트맥스(softmax) 확률 질량이 20% 이상 감소하고 검증 퍼플렉시티가 600-1000% 악화된다. 반면 같은 개수의 무작위 특성을 제거하면 확률은 최대 0.3% 감소하고 퍼플렉시티는 약 0.1% 악화될 뿐이다.

이런 극단적 이상치가 있는 상황에서 효과적인 양자화를 지원하기 위해, 우리 방법의 두 번째 부분인 **혼합 정밀도 분해(mixed-precision decomposition)**를 개발한다. 이상치 특성 차원에 대해서는 16비트 행렬 곱셈을, 나머지 99.9%의 차원에 대해서는 8비트 행렬 곱셈을 수행한다. 벡터 단위 양자화와 혼합 정밀도 분해의 결합을 **LLM.int8()**이라 이름 붙인다. LLM.int8()을 사용하면 최대 175B 파라미터의 LLM에서 어떤 성능 저하도 없이 추론할 수 있음을 보인다. 우리의 방법은 이 이상치들이 모델 성능에 미치는 영향에 대한 새로운 통찰을 제공할 뿐 아니라, OPT-175B/BLOOM 같은 초대형 모델을 소비자용 GPU를 갖춘 단일 서버에서 처음으로 사용할 수 있게 한다. 우리의 연구는 대형 언어 모델을 성능 저하 없이 접근 가능하게 만드는 데 초점을 두지만, 부록 D에서는 BLOOM-176B 같은 대형 모델의 종단간(end-to-end) 추론 런타임 성능이 유지되며 6.7B 파라미터 이상 GPT-3 규모 모델의 행렬 곱셈에서는 다소의 속도 향상도 제공됨을 함께 보인다. 우리는 소프트웨어를 오픈소스로 공개하고³, 선형 계층을 가진 모든 Hugging Face 호스팅 모델에서 이 방법을 쓸 수 있도록 Hugging Face Transformers 통합을 릴리스한다.

2. 나머지 파라미터는 대부분 임베딩 계층에서 나온다. 아주 적은 양이 norm과 바이어스(bias)에서 나온다.

3. <https://github.com/TimDettmers/bitsandbytes>

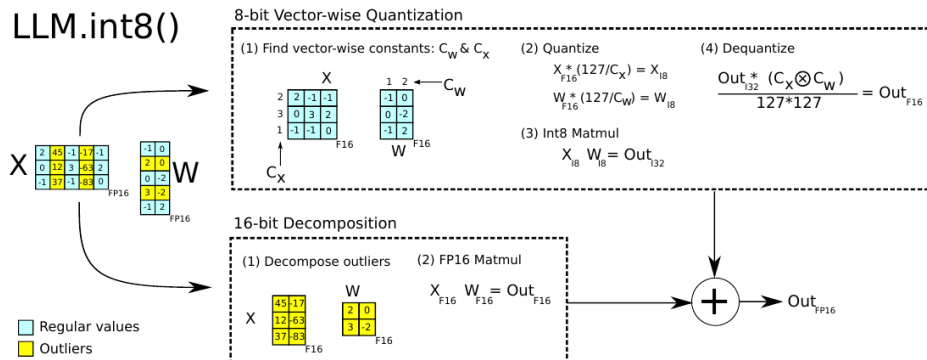


Figure 2: LLM.int8()의 개요도. 16비트 부동소수점 입력 X_{F16} 과 가중치 W_{F16} 이 주어지면, 특성과 가중치를 큰 크기의 특성(이상치)과 나머지 값의 부분 행렬들로 분해한다. 이상치 특성 행렬은 16비트로 곱한다. 나머지 모든 값은 8비트로 곱한다. 8비트 벡터 단위 곱셈은 행·열 방향 절대 최댓값 C_X, C_W 로 스케일링한 뒤 출력을 Int8로 양자화하여 수행한다. Int32 행렬 곱셈 출력 Out_{I32} 는 정규화 상수의 외적 $C_X \otimes C_W$ 로 역양자화(dequantize)한다. 마지막으로 이상치 출력과 일반 출력을 16비트 부동소수점 출력으로 합산한다.

2. 배경 (Background)

이 연구에서 우리는 Transformer 모델의 규모를 키움으로써 양자화 기법을 한계점까지 밀어붙인다. 우리는 두 가지 질문에 관심이 있다: 어떤 규모에서, 왜 양자화 기법이 실패하는가, 그리고 이것이 양자화 정밀도와 어떻게 연관

되는가? 이 질문에 답하기 위해 고정밀 비대칭 양자화인 「제로포인트 양자화(zeropoint quantization)」와 대칭 양자화인 「절대 최댓값 양자화(absolute maximum quantization, absmax)」를 연구한다. 제로포인트 양자화는 데이터 타입의 전체 비트 범위를 사용해 높은 정밀도를 제공하지만 실용적 제약 때문에 거의 쓰이지 않는다. 절대 최댓값 양자화가 가장 흔히 쓰이는 기법이다.

2.1 8비트 데이터 타입과 양자화 (8-bit Data Types and Quantization)

Absmax 양자화는 입력에 $s_{x_{f16}}$ 을 곱하여 8비트 범위 $[-127, 127]$ 로 스케일링한다. 여기서 $s_{x_{f16}}$ 은 127을 텐서 전체의 절대 최댓값으로 나눈 값이다. 이는 무한대 노름(infinity norm)으로 나눈 뒤 127을 곱하는 것과 같다. 따라서 FP16 입력 행렬 $X_{f16} \in \mathbb{R}^{s \times h}$ 에 대한 Int8 absmax 양자화는 다음과 같다:

$$X_{i8} = \lfloor 127 \cdot X_{f16} / \max_{ij}(|X_{f16,ij}|) \rfloor = \lfloor (127 / \|X_{f16}\|_{\infty}) X_{f16} \rfloor = \lfloor s_{x_{f16}} X_{f16} \rfloor$$

여기서 $\lfloor \cdot \rfloor$ 는 가장 가까운 정수로의 반올림을 나타낸다.

제로포인트 양자화는 정규화된 동적 범위(normalized dynamic range) nd_x 로 스케일링한 뒤 제로포인트 zp_x 만큼 이동시켜 입력 분포를 전체 범위 $[-127, 127]$ 로 옮긴다. 이 아핀(affine) 변환으로 어떤 입력 텐서든 데이터 타입의 모든 비트를 사용하게 되어 비대칭 분포에 대한 양자화 오차가 줄어든다. 예컨대 ReLU 출력의 경우 absmax 양자화에서는 $[-127, 0)$ 구간의 값이 전혀 쓰이지 않는 반면, 제로포인트 양자화에서는 전체 $[-127, 127]$ 범위가 쓰인다. 제로포인트 양자화는 다음 식으로 주어진다:

$$nd_{x_{f16}} = 2 \cdot 127 / (\max_{ij}(X_{f16}^{ij}) - \min_{ij}(X_{f16}^{ij}))$$

$$zp_{x_{i16}} = \lfloor X_{f16} \cdot \min_{ij}(X_{f16}^{ij}) \rfloor$$

$$X_{i8} = \lfloor nd_{x_{f16}} X_{f16} \rfloor$$

연산에서 제로포인트 양자화를 사용하려면 텐서 X_{i8} 과 제로포인트 $zp_{x_{i16}}$ 을 특수 명령어⁴에 함께 넘긴다. 이 명령어는 16비트 정수 연산을 수행하기 전에 X_{i8} 의 각 원소에 $zp_{x_{i16}}$ 을 더한다. 예를 들어 제로포인트 양자화된 두 수 A_{i8} 과 B_{i8} 을 각각의 제로포인트 $zp_{a_{i16}}, zp_{b_{i16}}$ 과 함께 곱하려면 다음을 계산한다:

$$C_{i32} = \text{multiply}_{i16}(A_{zp_{a_{i16}}}, B_{zp_{b_{i16}}}) = (A_{i8} + zp_{a_{i16}})(B_{i8} + zp_{b_{i16}})$$

GPU나 TPU처럼 multiply_{i16} 명령어가 없는 경우 다음처럼 전개(unrolling)해야 한다:

$$C_{i32} = A_{i8}B_{i8} + A_{i8}zp_{b_{i16}} + B_{i8}zp_{a_{i16}} + zp_{a_{i16}}zp_{b_{i16}}$$

여기서 $A_{i8}B_{i8}$ 은 Int8 정밀도로 계산되고 나머지는 Int16/32 정밀도로 계산된다. 따라서 multiply_{i16} 명령어가 없으면 제로포인트 양자화는 느릴 수 있다. 두 경우 모두 출력은 32비트 정수 C_{i32} 로 누적된다. C_{i32} 를 역양자화하려면 스케일링 상수 $nd_{a_{f16}}$ 과 $nd_{b_{f16}}$ 으로 나눈다.

16비트 부동소수점 입·출력을 갖는 Int8 행렬 곱셈. 시퀀스 차원 s , 특성 차원 h , 출력 차원 o 에 대해 은닉 상태 $X_{f16} \in \mathbb{R}^{s \times h}$ 와 가중치 $W_{f16} \in \mathbb{R}^{h \times o}$ 가 주어지면, 16비트 입·출력을 갖는 8비트 행렬 곱셈을 다음과 같이 수행한다:

$$X_{f16} W_{f16} = C_{f16} \approx (1 / c_{x_{f16}} c_{w_{f16}}) C_{i32} = S_{f16} \cdot C_{i32} \approx S_{f16} \cdot A_{i8} B_{i8} = S_{f16} \cdot Q(A_{f16}) Q(B_{f16}) \quad (6)$$

여기서 $Q(\cdot)$ 는 absmax 또는 제로포인트 양자화이고, $c_{x_{f16}}$ 과 $c_{w_{f16}}$ 은 각각의 텐서 단위 스케일링 상수로, absmax의 경우 s_x 와 s_w , 제로포인트의 경우 nd_x 와 nd_w 다.

3. 대규모에서의 Int8 행렬 곱셈 (Int8 Matrix Multiplication at Scale)

텐서당 하나의 스케일링 상수를 쓰는 양자화 방법의 주된 문제는, 단 하나의 이상치가 나머지 모든 값의 양자화 정밀도를 떨어뜨릴 수 있다는 것이다. 따라서 블록 단위 상수(block-wise constant)처럼 텐서당 여러 스케일링 상수를 두어 이상치의 영향을 각 블록 안에 가두는 것이 바람직하다. 우리는 양자화를 블록화하는 가장 흔한 방식 중 하나인 행 단위 양자화(row-wise quantization)를, 아래에서 자세히 설명하는 벡터 단위 양자화로 개선한다.

6.7B 규모를 넘어 모든 Transformer 계층에서 나타나는 큰 크기의 이상치 특성을 다루기에는 벡터 단위 양자화만으로 더 이상 충분하지 않다. 이를 위해 우리는 혼합 정밀도 분해를 개발한다. 큰 크기를 갖는 소수(약 0.1%)의 특성 차원은 16비트 정밀도로 표현하고, 나머지 99.9%의 값은 8비트로 곱한다. 대부분의 항목이 여전히 저정밀도로 표현되므로 16비트 대비 약 50%의 메모리 절감을 유지한다. 예컨대 BLOOM-176B에서는 모델의 메모리 사용량(footprint)을 1.96배 줄인다. 벡터 단위 양자화와 혼합 정밀도 분해는 Figure 2에 나타나 있다. LLM.int8() 방법은 absmax 벡터 단위 양자화와 혼합 정밀도 분해의 결합이다.

3.1 벡터 단위 양자화 (Vector-wise Quantization)

행렬 곱셈에서 스케일링 상수의 수를 늘리는 한 방법은 행렬 곱셈을 독립적인 내적들의 나열로 보는 것이다. 은닉 상태 $X_{f16} \in \mathbb{R}^{b \times h}$ 와 가중치 행렬 $W_{f16} \in \mathbb{R}^{h \times o}$ 가 주어지면, X_{f16} 의 각 행에 서로 다른 스케일링 상수 $c_{x_{f16}}$ 을, W_{f16} 의 각 열에 c_w 를 배정할 수 있다. 역양자화하려면 각 내적 결과를 $1/(c_{x_{f16}} c_{w_{f16}})$ 로 역정규화한다. 행렬 곱셈 전체로 보면 이는 외적 $c_{x_{f16}} \otimes c_{w_{f16}}$ 으로 역정규화하는 것과 동등하다($c_x \in \mathbb{R}^s, c_w \in \mathbb{R}^o$). 따라서 행·열 상수를 갖는 행렬 곱셈의 전체 식은 다음과 같다:

$$C_{f16} \approx (1 / (c_{x_{f16}} \otimes c_{w_{f16}})) C_{i32} = S \cdot C_{i32} = S \cdot A_{i8} B_{i8} = S \cdot Q(A_{f16}) Q(B_{f16})$$

우리는 이를 행렬 곱셈을 위한 벡터 단위 양자화라 부른다.

3.2 LLM.int8()의 핵심: 혼합 정밀도 분해 (The Core of LLM.int8(): Mixed-precision Decomposition)

우리의 분석은, 수십억 규모 8비트 Transformer의 중대한 문제가 큰 크기의 특성(열)을 갖고 있다는 점임을 보여 준다. 이 특성들은 Transformer 성능에 중요하며 높은 정밀도의 양자화를 요구한다. 그러나 우리의 최선의 양자화 기법인 벡터 단위 양자화는 은닉 상태의 각 행을 양자화하므로 이상치 특성에는 효과가 없다. 다행히 이 이상치 특성들은 실제로 대단히 희소하고 체계적이어서 전체 특성 차원의 약 0.1%만 차지하며, 이 덕분에 바로 그 차원들에 대해 고정밀 곱셈에 집중하는 새로운 분해 기법을 개발할 수 있다.

입력 행렬 $X_{f16} \in \mathbb{R}^{s \times h}$ 가 주어질 때, 이 이상치들은 거의 모든 시퀀스 차원 s 에서 체계적으로 발생하지만 특정한 특성/은닉 차원 h 에 국한됨을 발견했다. 따라서 우리는 행렬 곱셈을 위한 혼합 정밀도 분해를 제안한다. 임계값 α 보다 큰 크기의 이상치를 하나 이상 갖는 h 의 모든 차원을 담은 집합 $O = \{i \mid i \in \mathbb{Z}, 0 \leq i \leq h\}$ 로 이상치 특성 차원을 분리한다. 우리 연구에서는 $\alpha = 6.0$ 이면 Transformer의 성능 저하를 0에 가깝게 줄이기에 충분함을 발견했다. 모든 인덱스를 위 첨자로 쓰는 Einstein 표기법을 쓰면, 가중치 행렬 $W_{f16} \in \mathbb{R}^{h \times o}$ 에 대한 행렬 곱셈의 혼합 정밀도 분해는 다음과 같이 정의된다:

$$C_{f16} \approx \sum_{h \in O} X_{f16}^h W_{f16}^h + S_{f16} \cdot \sum_{h \notin O} X_{i8}^h W_{i8}^h$$

여기서 S_{f16} 은 Int8 입력·가중치 행렬 X_{i8} , W_{i8} 에 대한 역정규화 항이다.

이렇게 8비트와 16비트로 분리하면 이상치에 대해서는 고정밀 곱셈을 쓰면서, 99.9%가 넘는 값에 대해서는 8비트 가중치를 사용하는 메모리 효율적 행렬 곱셈을 쓸 수 있다. 13B 파라미터까지의 Transformer에서 이상치 특성 차원의 개수가 7을 넘지 않으므로($|O| \leq 7$), 이 분해 연산은 약 0.1%의 추가 메모리만 소비한다.

3.3 실험 설정 (Experimental Setup)

우리는 공개된 여러 사전학습 언어 모델의 크기를 최대 175B 파라미터까지 키워 가며 양자화 방법의 견고함(robustness)을 측정한다. 핵심 질문은 특정 모델에서 어떤 양자화 방법이 얼마나 잘 작동하느냐가 아니라, 규모를 키울 때 그 방법의 성능이 어떤 추세를 보이느냐다.

우리는 실험에 두 가지 설정을 사용한다. 하나는 언어 모델링 퍼플렉시티에 기반한 것으로, 우리는 이것이 양자화 저하에 매우 민감한 대단히 견고한 척도임을 발견했다. 이 설정으로 여러 양자화 기준선을 비교한다. 추가로 OPT 모델에 대해 다양한 최종 과제(end task)에서의 제로샷 정확도 저하를 평가하며, 여기서는 16비트 기준선과 우리 방법을 비교한다.

언어 모델링 설정에는 fairseq로 사전학습된 125M-13B 파라미터의 밀집(dense) 자기회귀(autoressive) Transformer를 사용한다. 이 Transformer들은 Books, English Wikipedia, CC-News, OpenWebText, CC-Stories, English CC100으로 사전학습되었다. 이 사전학습 모델들이 어떻게 학습되었는지는 Artetxe 등(2021)을 참조하라.

Int8 양자화 후의 언어 모델링 저하를 평가하기 위해, Common Crawl 코퍼스의 부분집합인 C4 코퍼스의 검증 데이터에서 8비트 Transformer의 퍼플렉시티를 평가한다. 이 평가에는 NVIDIA A40 GPU를 사용한다.

제로샷 성능의 저하를 측정하기 위해 OPT 모델을 사용하며, EleutherAI 언어 모델 평가 하네스(evaluation harness)로 평가한다.

3.4 주요 결과 (Main Results)

C4 코퍼스에서 평가한 125M-13B Int8 모델의 주요 언어 모델링 퍼플렉시티 결과는 Table 1에 있다. absmax, 행 단위, 제로포인트 양자화는 규모가 커지면 실패하며, 2.7B 파라미터를 넘으면 모델이 더 작은 모델보다 나쁜 성능을 보인다. 제로포인트 양자화는 그 대신 6.7B 파라미터를 넘어서 실패한다. 우리의 방법 LLM.int8()만이 퍼플렉시티를 보존하는 유일한 방법이며, 따라서 LLM.int8()만이 유리한 확장 추세를 갖는다.

Parameters	125M	1.3B	2.7B	6.7B	13B
32-bit Float	25.65	15.91	14.43	13.30	12.45
Int8 absmax	87.76	16.55	15.11	14.59	19.08
Int8 zeropoint	56.66	16.24	14.76	13.49	13.94
Int8 absmax row-wise	30.93	17.08	15.24	14.13	16.49
Int8 absmax vector-wise	35.84	16.82	14.98	14.13	16.48
Int8 zeropoint vector-wise	25.72	15.94	14.36	13.38	13.47
Int8 absmax row-wise + decomposition	30.76	16.19	14.65	13.25	12.46
Absmax LLM.int8() (vector-wise + decomp)	25.83	15.93	14.44	13.24	12.45
Zeropoint LLM.int8() (vector-wise + decomp)	25.69	15.92	14.43	13.24	12.45

Table 1: 125M-13B 파라미터에 이르는 다양한 크기의 Transformer에 대한 양자화 방법별 C4 검증 퍼플렉시티. absmax, 행 단위, 제로포인트, 벡터 단위 양자화는 규모가 커질수록 상당한 성능 저하를 일으키며, 특히 13B 지점에서는 8비트 13B의 퍼플렉시티가 8비트 6.7B보다 나쁘다. LLM.int8()을 쓰면 규모가 커져도 완전한 퍼플렉시티를 회복한다. 제로포인트 양자화는 비대칭 양자화 덕에 이점을 보이지만, 혼합 정밀도 분해와 함께 쓰면 그 이점이 사라진다.

EleutherAI 평가 하네스에서 OPT 모델의 제로샷 성능 확장 추세를 보면(Figure 1), LLM.int8()은 125M에서 175B 파라미터로 확장하는 동안 완전한 16비트 성능을 유지한다. 반면 기준선인 8비트 absmax 벡터 단위 양자화는 확장성이 나빠 무작위 수준의 성능으로 퇴화한다.

우리의 일차적 초점은 메모리 절감이지만, LLM.int8()의 실행 시간도 측정했다. 양자화 오버헤드는 6.7B 파라미터 미만의 모델에서 FP16 기준선 대비 추론을 느리게 할 수 있다. 그러나 6.7B 파라미터 이하 모델은 대부분의 GPU에 들어가므로 실무에서는 양자화의 필요성이 낮다. LLM.int8()의 실행 시간은 175B 모델의 것과 동등한 대형 행렬 곱셈에서 약 2배 빠르다. 부록 D에 이 실험들의 자세한 내용이 있다.

4. 대규모 Transformer에서 창발하는 큰 크기의 특성 (Emergent Large Magnitude Features in Transformers at Scale)

Transformer의 규모를 키우면 큰 크기의 이상치 특성이 창발하여 모든 계층과 그 양자화에 강하게 영향을 미친다. s 를 시퀀스/토큰 차원, h 를 은닉/특성 차원이라 할 때 은닉 상태 $X \in \mathbb{R}^{s \times h}$ 가 주어지면, 우리는 특성을 특정 차원 h_i 로 정의한다. 우리의 분석은 주어진 Transformer의 모든 계층에 걸쳐 특정 특성 차원 h_i 를 살펴본다.

우리는 이상치 특성이 어텐션과 Transformer의 전반적 예측 성능에 강하게 영향을 미침을 발견했다. 13B 모델에서 2048 토큰 시퀀스당 최대 15만 개의 이상치가 존재하지만, 이 이상치 특성들은 매우 체계적이어서 기껏해야 7개의 고유한 특성 차원 h_i 만을 차지한다. 이 분석에서 얻은 통찰은 혼합 정밀도 분해를 개발하는 데 결정적이었다. 우리의 분석은 제로포인트 양자화가 갖는 이점과 그 이점이 혼합 정밀도 분해를 쓰면 사라지는 이유, 그리고 작은 모델과 큰 모델의 양자화 성능 차이를 설명한다.

4.1 이상치 특성 찾기 (Finding Outlier Features)

창발 현상(emergent phenomena)의 정량적 분석이 어려운 이유는 두 가지다. 분석 결과가 이해 가능하고 지나치게 복잡하지 않도록 소수의 특성만 골라야 하는 동시에, 중요한 확률적·구조적 패턴을 놓치지 않아야 한다. 우리는 경험적 접근으로 이 제약을 찾는다. 다음 기준에 따라 이상치를 정의한다: 특성의 크기가 최소 6.0이고, 전체 계층의 최소 25%에 영향을 주며, 시퀀스 차원의 최소 6%에 영향을 준다.

더 형식적으로, L 개 계층과 은닉 상태 $X_l \in \mathbb{R}^{s \times h}$, $l = 0 \dots L$ 을 갖는 Transformer가 주어질 때(s 는 시퀀스 차원, h 는 특성 차원), 우리는 특성을 은닉 상태 X_l 중 어느 하나의 특정 차원 h_i 로 정의한다. 크기 $\alpha \geq 6$ 인 값을 하나 이상 갖는 차원 h_i ($0 \leq i \leq h$)를 추적하되, 이 이상치가 Transformer 계층 $0 \dots L$ 의 최소 25%에서 같은 특성 차원 h_i 에 나타나고 모든 은닉 상태 X_l 에 걸쳐 전체 시퀀스 차원 s 의 최소 6%에 나타나는 경우에만 통계를 수집한다. 특성 이상치는 어텐션 투영(key/query/value/output)과 피드포워드 네트워크의 확장 계층(첫 번째 부속 계층)에서만 발생하므로, 이 분석에서는 어텐션 함수 자체와 FFN 축소 계층(두 번째 부속 계층)은 무시한다.

이 임계값들에 대한 우리의 논거는 다음과 같다. 혼합 정밀도 분해를 사용할 때 크기 6 이상의 특성을 모두 이상치 특성으로 취급하면 퍼플렉시티 저하가 멈춘다는 것을 발견했다. 이상치의 영향을 받는 계층 수의 경우, 이상치 특성이 큰 모델에서는 체계적임을 발견했다: 대부분의 계층에서 나타나거나 아예 나타나지 않는다. 반면 작은 모델에서는 확률적이다: 시퀀스마다 일부 계층에서 가끔 나타난다. 따라서 우리는 가장 작은 125M 모델에서 이상치가 단 하

나만 검출되도록 계층 임계값을 정했다. 이는 전체 Transformer 계층의 최소 25%가 같은 특성 차원의 이상치에 영향을 받는 조건에 해당한다. 두 번째로 혼한 이상치는 단 하나의 계층(계층의 2%)에서만 나타나므로 이것이 합리적인 임계값임을 시사한다. 같은 절차로 125M 모델에서 이상치 특성의 영향을 받는 시퀀스 차원 비율의 임계값을 찾았다: 이상치는 시퀀스 차원의 최소 6%에서 발생한다.

우리는 13B 파라미터 규모까지의 모델을 시험한다. 관찰된 현상이 소프트웨어 버그 때문이 아님을 확실히 하기 위해, 세 가지 다른 소프트웨어 프레임워크에서 학습된 Transformer를 평가한다. OpenAI 소프트웨어를 쓰는 GPT-2 모델 4개, Fairseq를 쓰는 Meta AI 모델 5개, Tensorflow-Mesh를 쓰는 EleutherAI 모델 GPT-J 1개를 평가한다. 자세한 내용은 부록 C에 있다. 또한 두 가지 다른 추론 소프트웨어 프레임워크(Fairseq, Hugging Face Transformers)에서도 분석을 수행한다.

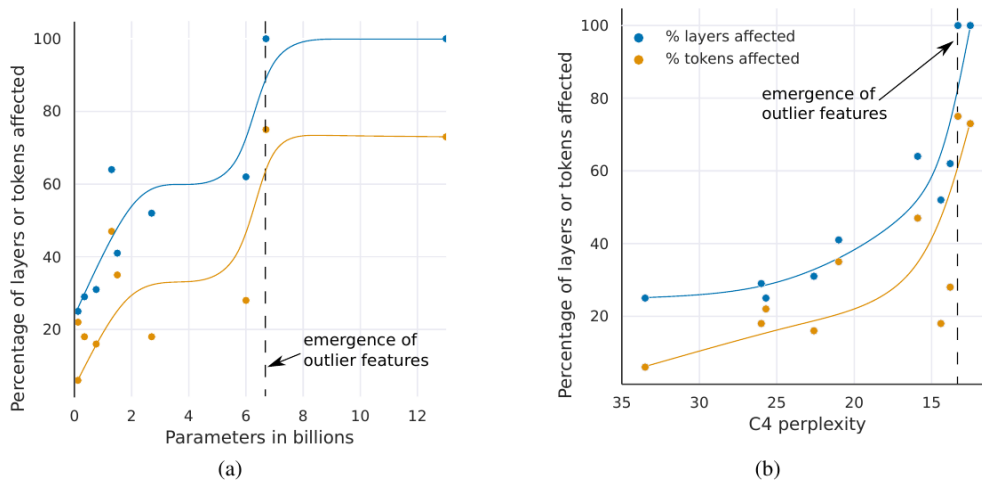


Figure 3: (a) 모델 크기 또는 (b) C4 퍼플렉시티에 따른, 큰 크기의 이상치 특성이 영향을 미치는 Transformer 계층 및 전체 시퀀스 차원의 비율. 곡선은 (a)는 4개, (b)는 9개의 선형 구간을 갖는 B-스플라인 보간이다. 상전이가 일어나면 이상치는 모든 계층과 전체 시퀀스 차원의 약 75%에 존재한다. (a)는 파라미터 크기에 대한 급격한 상전이를 시사하는 반면, (b)는 퍼플렉시티가 감소함에 따른 점진적·지수적 상전이를 시사한다. (a)의 뚜렷한 변화는 양자화 방법의 성능이 갑자기 저하되는 지점과 함께 일어난다.

4.2 이상치 특성의 효과 측정 (Measuring the Effect of Outlier Features)

이상치 특성이 어텐션과 예측 성능에 필수적임을 입증하기 위해, 은닉 상태 x_l 을 어텐션 투영 계층에 넣기 전에 이상치 특성을 0으로 만든 뒤, top-1 소프트맥스 확률을 이상치가 있는 통상적 소프트맥스 확률과 비교한다. 이를 모든 계층에 대해 독립적으로 수행한다. 즉 연쇄 오차(cascading error)를 피하고 이상치 특성만의 효과를 분리하기 위해, 뒤 계층으로는 통상적 소프트맥스 확률 값을 전달한다. 또한 이상치 특성 차원을 제거(0으로 설정)하고 이 변경된 은닉 상태를 Transformer에 전파했을 때의 퍼플렉시티 저하도 보고한다. 대조군으로, 이상치가 아닌 무작위 특성 차원에 같은 절차를 적용하고 어텐션·퍼플렉시티 저하를 기록한다.

우리의 주요 정량적 결과는 네 가지로 요약된다.

(1) 파라미터 수로 측정하면, Transformer의 모든 계층에 걸친 큰 크기 특성의 창발은 Figure 3a에서 보듯 6B와 6.7B 파라미터 사이에서 갑자기 일어나며, 영향을 받는 계층의 비율이 65%에서 100%로 증가한다. 영향을 받는 시퀀스 차원의 비율은 35%에서 75%로 급격히 늘어난다. 이 급격한 변화는 양자화가 실패하기 시작하는 지점과 함께 일어난다.

(2) 반면 퍼플렉시티로 측정하면, Transformer 전체 계층에 걸친 큰 크기 특성의 창발은 Figure 3b에서 보듯 퍼플렉시티 감소의 지수 함수에 따라 매끄럽게 나타나는 것으로 볼 수 있다. 이는 창발에 갑작스러운 것이 없으며, 더 작은 모델에서 지수적 추세를 연구하면 상전이가 일어나기 전에 창발적 특성을 탐지할 수 있을지도 모른다는 것을 시사한다. 또한 창발이 모델 크기만의 문제가 아니라, 사용된 학습 데이터의 양과 데이터 품질 같은 여러 추가 요인과 관련된 퍼플렉시티의 문제임을 시사한다.

(3) Figure 4a에서 보듯, 이상치 특성이 Transformer의 모든 계층에서 나타나기 시작하면 이상치 특성의 중앙값 크기가 급격히 커진다. 이상치 특성의 큰 크기와 그 비대칭 분포는 Int8 양자화 정밀도를 붕괴시킨다. 이것이 6.7B 규모부터 양자화 방법이 실패하는 핵심 이유다. 양자화 분포의 범위가 지나치게 커져 대부분의 양자화 구간(bin)이 비게 되고, 작은 양자화 값들은 0으로 양자화되어 사실상 정보가 소멸한다. 우리는 Int8 추론뿐 아니라, 6.7B 규모를 넘어서면 통상적 16비트 부동소수점 학습도 이상치 때문에 불안정해진다고 가설을 세운다. 크기 60의 값들로 채워진 벡터를 곱하다 보면 16비트 최댓값 65535를 우연히 초과하기 쉽기 때문이다.

(4) Figure 4b에서 보듯 이상치 특성의 개수는 C4 퍼플렉시티 감소에 대해 엄격하게 단조적으로(strictly monotonically) 증가하는 반면, 모델 크기와와의 관계는 비단조적이다. 이는 상전이를 결정하는 것이 단순한 모델 크기가 아니라 모델 퍼플렉시티임을 나타낸다. 우리는 모델 크기가 창발에 도달하는 데 필요한 여러 공변량(covariate) 중 하나일 뿐이라고 가설을 세운다.

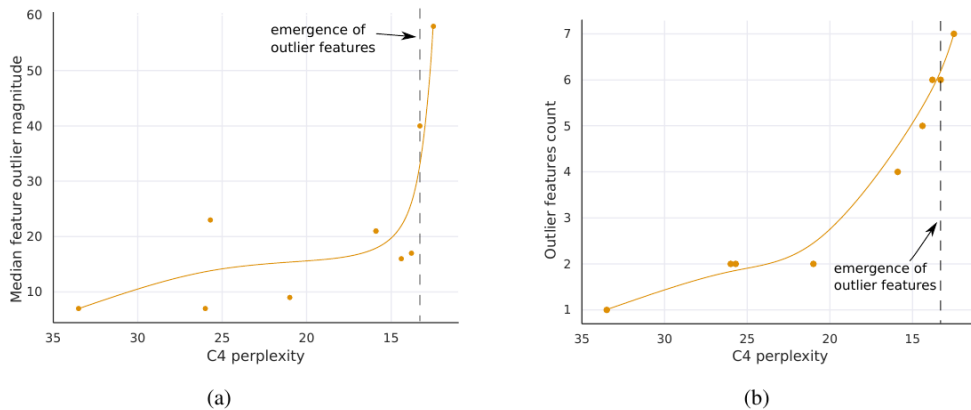


Figure 4: The median magnitude of the largest outlier feature in (a) indicates a sudden shift in outlier

Figure 4: (a)의 가장 큰 이상치 특성의 중앙값 크기는 이상치 크기의 급격한 변화를 나타낸다. 이것이 창발 이후 양자화 방법이 실패하는 주된 이유로 보인다. 이상치 특성 차원의 개수는 모델 크기에 대략 비례할 뿐이지만, (b)는 분석한 모든 모델에서 이상치 개수가 퍼플렉시티에 대해 엄격하게 단조적임을 보여준다. 곡선은 9개의 선형 구간을 갖는 B-스플라인 보간이다.

이 이상치 특성들은 상전이가 일어난 뒤에는 매우 체계적이다. 예컨대 시퀀스 길이 2048의 6.7B Transformer에서는 Transformer 전체에서 시퀀스당 약 15만 개의 이상치 특성이 발견되지만, 이들은 단 6개의 서로 다른 은닉 차원에 집중되어 있다.

이 이상치들은 Transformer 성능에 결정적이다. 이상치를 제거하면 이상치 특성 차원이 기껏해야 7개임에도 평균 top-1 소프트맥스 확률이 약 40%에서 약 20%로 줄고, 검증 퍼플렉시티가 600-1000% 증가한다. 대신 무작위 특성 차원 7개를 제거하면 top-1 확률은 0.02-0.3%만 줄고 퍼플렉시티는 0.1% 증가한다. 이는 이 특성 차원들의 결정적 성격을 부각한다. 아주 작은 오차도 모델 성능에 크게 영향을 미치므로, 이 이상치 특성에 대한 양자화 정밀도가 무엇보다 중요하다.

4.3 양자화 성능의 해석 (Interpretation of Quantization Performance)

우리의 분석은 특정 특성 차원의 이상치가 대형 Transformer에 편재하며, 이 특성 차원들이 Transformer 성능에 결정적임을 보여준다. 행 단위 및 벡터 단위 양자화는 은닉 상태의 시퀀스 차원 s (행)를 각각 스케일링하는데 이상치는 특성 차원 h (열)에서 발생하므로, 두 방법 모두 이 이상치를 효과적으로 다룰 수 없다. 이것이 absmax 양자화 방법이 창발 이후 빠르게 실패하는 이유다.

그러나 거의 모든 이상치는 엄격한 비대칭 분포를 갖는다: 오로지 양수이거나 오로지 음수다(부록 C 참조). 이 때문에 제로포인트 양자화가 이 이상치들에 특히 효과적이다. 제로포인트 양자화는 이 이상치들을 전체 $[-127, 127]$ 범위로 스케일링하는 비대칭 양자화 방법이기 때문이다. 이는 Table 1의 양자화 확장 벤치마크에서 제로포인트가 강한 성능을 보인 것을 설명한다. 그러나 13B 규모에서는 누적된 양자화 오차와 이상치 크기의 빠른 증가(Figure 4a) 때문에 제로포인트 양자화조차 실패한다.

혼합 정밀도 분해를 포함한 완전한 LLM.int8() 방법을 쓰면 제로포인트 양자화의 이점이 사라지는데, 이는 분해되고 남은 특성들이 대칭적임을 나타낸다. 다만 벡터 단위는 여전히 행 단위 양자화보다 이점이 있으며, 이는 완전한 정밀도의 예측 성능을 유지하려면 모델 가중치에 대한 향상된 양자화 정밀도가 필요함을 나타낸다.

5. 관련 연구 (Related Work)

양자화 데이터 타입과 Transformer의 양자화에 대한 밀접한 관련 연구가 아래에 기술되어 있다. 합성곱(convolutional) 신경망의 양자화에 대한 추가 관련 연구는 부록 B에 있다.

8비트 데이터 타입. 우리의 연구는 Int8 데이터 타입을 둘러싼 양자화 기법을 다루는데, 현재 GPU가 지원하는 유일한 8비트 데이터 타입이 Int8이기 때문이다. 다른 흔한 데이터 타입으로는 고정소수점 또는 8비트 부동소수점(FP8) 데이터 타입이 있다. 이 데이터 타입들은 보통 부호 비트 1개와 서로 다른 지수/가수(fraction) 비트 조합을 갖는다. 예컨대 흔한 변형은 지수 5비트, 가수 2비트로, 스케일링 상수를 쓰지 않거나 제로포인트 스케일링을 쓴다. 이런 데이터 타입은 가수가 2비트뿐이어서 큰 크기의 값에 대해 큰 오차를 갖지만, 작은 크기의 값에는 높은 정확도를 제공한다. Jin 등(2022)은 특정 표준편차를 갖는 입력에 대해 어떤 고정소수점 지수/가수 비트 폭이 최적인지에 대한 훌륭한 분석을 제공한다. 우리는 FP8 데이터 타입이 Int8보다 우수한 성능을 제공한다고 믿지만, 현재 GPU도 TPU도 이 데이터 타입을 지원하지 않는다.

언어 모델의 이상치 특성. 언어 모델의 큰 크기 이상치 특성은 이전에도 연구된 바 있다. 선행 연구는 Transformer에서의 이상치 등장과 계층 정규화(layer normalization) 및 토큰 빈도 분포와의 이론적 관계를 증명했다. 유사하게 Kovaleva 등(2021)은 BERT 모델 계열의 이상치 등장을 LayerNorm에 귀속시키고, Puccetti 등(2022)은 이상치의 창발이 학습 분포의 토큰 빈도와 관련됨을 실증적으로 보였다. 우리는 자기회귀 모델의 규모가 이 이상치 특성의 창발적 속성과 어떻게 관련되는지 보이고, 이상치를 적절히 모델링하는 것이 효과적 양자화에 결정적임을 보임으로써 이 연구들을 확장한다.

수십억 규모 Transformer 양자화. 우리와 병행하여 개발된 두 방법이 있다: nuQmm과 ZeroQuant. 둘 다 같은 양자화 방식인 그룹 단위 양자화(group-wise quantization)를 사용하는데, 이는 벡터 단위 양자화보다 더 세밀한 양자화 정규화 상수 단위를 갖는다. 이 방식은 더 높은 양자화 정밀도를 제공하지만 맞춤 CUDA 커널도 요구한다. nuQmm과 ZeroQuant는 추론 가속과 메모리 사용량 감소를 목표로 하는 반면, 우리는 8비트 메모리 사용량 하에서 예측 성능을 보존하는 데 집중한다. nuQmm과 ZeroQuant가 평가한 가장 큰 모델은 각각 2.7B와 20B 파라미터 Transformer다. ZeroQuant는 20B 모델의 8비트 양자화에서 무손실 성능을 달성한다. 우리는 우리의 방법이 최대 176B 파라미터 모델의 무손실 양자화를 가능하게 함을 보인다. nuQmm과 ZeroQuant는 모두 더 세밀한 양자화 단위가 대형 모델을 양자화하는 효과적 수단일 수 있음을 시사하며, 이 방법들은 LLM.int8()과 상호 보완적이

다. 또 다른 병행 연구인 GLM-130B는 우리 연구의 통찰을 활용해 무손실 8비트 양자화를 달성한다. GLM-130B는 8비트로 가중치를 저장하되 행렬 곱셈은 완전한 16비트 정밀도로 수행한다.

6. 논의와 한계 (Discussion and Limitations)

우리는 수십억 파라미터 Transformer를 Int8로 양자화하여 성능 저하 없이 즉시 추론에 사용할 수 있음을 처음으로 입증했다. 대규모에서 창발하는 큰 크기의 특성을 분석해 얻은 통찰로, 이상치 특성을 별도의 16비트 행렬 곱셈으로 분리하는 혼합 정밀도 분해를 개발함으로써 이를 달성한다. 벡터 단위 양자화와 결합하면 우리의 방법 LLM.int8()이 되며, 최대 175B 파라미터 모델의 완전한 추론 성능을 회복할 수 있음을 실증적으로 보였다.

이 연구의 주된 한계는 분석이 오로지 Int8 데이터 타입에 대한 것이며 8비트 부동소수점(FP8) 데이터 타입은 연구하지 않았다는 점이다. 현재 GPU와 TPU가 이 데이터 타입을 지원하지 않으므로 이는 향후 연구로 남기는 것이 최선이라 본다. 다만 Int8 데이터 타입에서 얻은 많은 통찰이 FP8 데이터 타입으로도 직접 이어질 것이라 믿는다. 또 다른 한계는 최대 175B 파라미터 모델만 연구했다는 점이다. 우리가 175B 모델을 성능 저하 없이 Int8로 양자화하긴 했지만, 더 큰 규모에서는 추가적인 창발적 속성이 우리의 양자화 방법을 무너뜨릴 수도 있다.

세 번째 한계는 어텐션 함수에 Int8 곱셈을 쓰지 않았다는 점이다. 우리의 초점이 메모리 사용량 감소에 있고 어텐션 함수는 파라미터를 쓰지 않으므로 이는 엄밀히 필요하지 않았다. 다만 이 문제에 대한 초기 탐색은 여기서 개발한 것 이상의 추가적인 양자화 방법이 필요함을 시사했고, 우리는 이를 향후 연구로 남긴다.

마지막 한계는 추론에 집중하고 학습이나 미세조정(finetuning)은 연구하지 않았다는 점이다. 대규모 Int8 미세조정과 학습에 대한 초기 분석은 부록 E에 제공한다. 대규모 Int8 학습은 양자화 정밀도, 학습 속도, 엔지니어링 복잡도 사이의 까다로운 절충을 요구하는 매우 어려운 문제이며, 역시 향후 연구로 남긴다.

Class	Hardware	GPU Memory	8-bit	16-bit
Enterprise	8x A100	80 GB	OPT-175B / BLOOM	OPT-175B / BLOOM
Enterprise	8x A100	40 GB	OPT-175B / BLOOM	OPT-66B
Academic server	8x RTX 3090	24 GB	OPT-175B / BLOOM	OPT-66B
Academic desktop	4x RTX 3090	24 GB	OPT-66B	OPT-30B
Paid Cloud	Colab Pro	15 GB	OPT-13B	GPT-J-6B
Free Cloud	Colab	12 GB	T0/T5-11B	GPT-2 1.3B

Table 2: 다양한 하드웨어 구성에서 16비트와 8비트 정밀도로 각각 실행할 수 있는 가장 큰 모델. 우리의 8비트 방법이 이전에는 접근 불가능했던 많은 모델, 특히 OPT-175B/BLOOM을 접근 가능하게 만들 수 있다.

7. 더 넓은 영향 (Broader Impacts)

우리 연구의 주된 영향은 이전에는 GPU 메모리에 들어가지 않던 대형 모델에 대한 접근을 가능하게 한다는 것이다. 이는 특히 자원이 가장 부족한 연구자들에게, 제한된 GPU 메모리 때문에 이전에는 불가능했던 연구와 응용을 가능하게 한다. 성능 저하 없이 이제 접근 가능해진 모델/GPU 조합은 Table 3을 보라. 그러나 우리의 연구는 많은 GPU를 가진 자원이 풍부한 조직이 같은 수의 GPU로 더 많은 모델을 서비스할 수 있게도 하며, 이는 자원이 풍부한 조직과 부족한 조직 사이의 격차를 키울 수 있다.

특히 우리는 대형 사전학습 모델의 공개 릴리스(예: 최근의 Open Pretrained Transformers, OPT)가 우리의 새로운 제로샷·퓨샷 프롬프팅용 Int8 추론과 결합하여, 자원 제약 때문에 이전에는 불가능했던 학술 기관의 새로운 연구를 가능하게 하리라 믿는다. 이런 대규모 모델의 광범위한 접근성은 예측하기 어려운 이로인 효과와 해로운 효과를 모두 사회에 미칠 가능성이 높다.

감사의 글 (Acknowledgments) 이 연구의 초안에 피드백을 준 Ofir Press, Gabriel Ilharco, Daniel Jiang, Mitchell Wortsman, Ari Holtzman, Mitchell Gordon에게 감사한다. Hugging Face Transformers 통합을 도와준 JustHeuristic (Yozh)과 Titus von K  ller에게 감사한다.

참고문헌 (References)

참고문헌은 저자·서지 정보의 나열이므로 번역하지 않는다. 원문 arXiv:2208.07339의 11-14쪽을 참조하라. NeurIPS 체크리스트 (원문 14-15쪽)도 같은 이유로 생략한다.

부록 A. 16비트 정밀도 대비 메모리 사용량 (Memory Usage Compared to 16-bit Precision)

Table 3은 다양한 오픈소스 모델에 대해 16비트 추론과 LLM.int8()의 메모리 사용량을 비교한다. LLM.int8()이 가장 큰 오픈소스 모델인 OPT-175B와 BLOOM-176B를 소비자용 GPU를 장착한 단일 노드에서 실행할 수 있게 함을 볼 수 있다.

Table 3: 다양한 하드웨어 구성에서 16비트와 8비트 정밀도로 실행할 수 있는 방법. 본문 Table 2와 동일한 표이므로 다시 실지 않는다. 우리의 8비트 방법이 이전에는 접근 불가능했던 많은 모델, 특히 OPT-175B/BLOOM을 접근 가능하게 만든다.

부록 B. 추가 관련 연구 (Additional Related Work)

1B 파라미터 미만 Transformer의 양자화. Transformer의 양자화는 BERT와 RoBERTa를 포함한 10억 파라미터 미만의 마스크 언어 모델(masked language model, MLM)에 집중되어 왔다. 8비트 BERT/RoBERTa의 변형으로는 Q8BERT, QBERT, 양자화 노이즈를 쓰는 곱 양자화(product quantization), TernaryBERT, BinaryBERT가 있다. Zhao 등(2021)의 연구는 양자화와 가지치기(pruning)를 함께 수행한다. 이 모델들은 모두 저정밀도에서 모델을 쓸 수 있게 만들기 위해 양자화 인지 미세조정(quantization-aware finetuning)이나 학습 후 양자화(post-training quantization)를 요구한다. 이와 대조적으로 우리의 방법에서는 모델을 성능 저하 없이 곧바로 사용할 수 있다.

행렬 곱셈을 1×1 합성곱으로 본다면, 벡터 단위 양자화는 합성곱의 채널 단위 양자화(channel-wise quantization)를 행 양자화와 결합한 것과 동등하다. 행렬 곱셈에 대해서는 Wu 등(2020)이 BERT 크기(350M 파라미터)의 Transformer에 이를 사용했으며, 자기회귀·대규모 모델에 대한 벡터 단위 양자화를 연구한 것은 우리가 처음이다. BERT 외의 Transformer를 양자화한 우리가 아는 유일한 다른 연구는 Chen 등(2020)으로, 순전파에는 제로포인트 양자화, 역전파에는 제로포인트-행 단위 양자화를 쓰는 학습 후 양자화를 사용한다. 그러나 이 연구도 여전히 10억 파라미터 미만 Transformer에 대한 것이다. 우리는 평가에서 제로포인트와 행 단위 양자화 모두와 비교하며, 학습 후 양자화를 요구하지 않는다.

저비트폭 및 합성곱 신경망 양자화. 8비트 미만의 데이터 타입을 쓰는 연구는 보통 합성곱 신경망(CNN)을 위한 것으로, 모바일 기기에서 메모리 사용량을 줄이고 추론 속도를 높이면서 모델 저하를 최소화하려 한다. 다양한 비트폭의 방법들이 연구되었다: 1비트 방법, 2-3비트, 4비트, 그 이상의 비트, 또는 가변 비트. 추가 관련 연구는 Qin 등(2020)의 서베이를 보라. 수십억 규모 Transformer에서 어느 정도의 성능 저하를 감수한 8비트 미만 폭도 가능하다고 믿지만, 우리는 성능 저하가 없으면서 Int8 텐서 코어로 추론을 가속하는 널리 쓰이는 GPU의 혜택을 받을 수 있는 8비트 Transformer에 집중한다.

합성곱 신경망 양자화의 또 다른 연구 방향은 양자화 오차를 개선하기 위해 양자화 절차에 대한 조정을 학습하는 것이다. 예컨대 Hessian 정보 사용, 스텝 크기 양자화(step-size quantization), 소프트 양자화(soft quantization), 선형 계획 최적화를 통한 혼합 정밀도, 그 밖의 학습된 양자화 방법들이 있다.

부록 C. 이상치 특성 상세 데이터 (Detailed Outlier Feature Data)

Table 4는 우리의 이상치 특성 분석에서 얻은 데이터를 표로 제공한다. 각 Transformer에서 가장 흔한 이상치의 사분위수와, 한쪽으로 치우친(one-sided) 이상치, 즉 0을 가로지르지 않는 비대칭 분포를 갖는 이상치의 개수를 제공한다.

Model	PPL ↓	Params	Count	1-sided	Layers	SDims	Quartiles	w/ Outlier	No Outlier
GPT2	33.5	117M	1	1	25%	6%	(-8, -7, -6)	45%	19%
GPT2	26.0	345M	2	1	29%	18%	(6, 7, 8)	45%	19%
FSEQ	25.7	125M	2	2	25%	22%	(-40, -23, -11)	32%	24%
GPT2	22.6	762M	2	0	31%	16%	(-9, -6, 9)	41%	18%
GPT2	21.0	1.5B	2	1	41%	35%	(-11, -9, -7)	41%	25%
FSEQ	15.9	1.3B	4	3	64%	47%	(-33, -21, -11)	39%	15%
FSEQ	14.4	2.7B	5	5	52%	18%	(-25, -16, -9)	45%	13%
GPT-J	13.8	6.0B	6	6	62%	28%	(-21, -17, -14)	55%	10%
FSEQ	13.3	6.7B	6	6	100%	75%	(-44, -40, -35)	35%	13%
FSEQ	12.5	13B	7	6	100%	73%	(-63, -58, -45)	37%	16%

Table 4: 전체 계층의 최소 25%와 전체 시퀀스 차원의 최소 6%에서 발생하는, 크기 6 이상 이상치의 요약 통계. C4 검증 퍼플렉시티가 낮을수록 더 많은 이상치가 존재함을 볼 수 있다. 이상치는 보통 한쪽으로 치우쳐(one-sided) 있으며, 최대 범위를 보여주는 사분위수는 이상치의 크기가 보통 [-3.5, 3.5] 범위를 갖는 다른 특성 차원의 최대 크기보다 3-20배 큼을 나타낸다. 규모가 커질수록 이상치는 Transformer의 모든 계층에서 점점 더 흔해지고 거의 모든 시퀀스 차원에서 발생한다. 6.7B 파라미터에서 상전이(transition)가 일어나, 같은 이상치가 모든 계층의 같은 특성 차원에서 전체 시퀀스 차원(SDim)의 약 75%에 대해 발생한다. 전체 특성의 약 0.1%에 불과함에도 이 이상치들은 큰 소프트맥스 확률에 필수적이다. 이상치를 제거하면 평균 top-1 소프트맥스 확률이 약 20%p 줄어든다. 이상치가 시퀀스 차원 s에 걸쳐 대체로 비대칭 분포를 갖기 때문에, 이 이상치 차원들은 대칭적 absmax 양자화를 붕괴시키고 비대칭적 제로포인트 양자화에 유리하게 작용한다. 이는 우리의 검증 퍼플렉시티 분석 결과를 설명한다. 이 관찰은 서로 다른 소프트웨어 프레임워크(fairseq, OpenAI, Tensorflow-mesh)로 학습된 모델과 서로 다른 추론 프레임워크(fairseq, Hugging Face Transformers)에서 모두 발생하므로 보편적인 것으로 보인다. 또한 이 이상치들은 Transformer 아키텍처의 사소한 변형(로터리 임베딩, 임베딩 norm, 잔차 스케일링, 다른 초기화)에도 견고해 보인다.

부록 D. 추론 속도 향상과 저하 (Inference Speedups and Slowdowns)

D.1 행렬 곱셈 벤치마크 (Matrix Multiplication Benchmarks)

우리의 연구는 모델을 접근 가능하게 만들기 위한 메모리 효율에 초점을 두지만, Int8 방법은 추론 가속에도 자주 쓰인다. 우리는 양자화 분해 오버헤드가 상당하며, Int8 행렬 곱셈 자체는 GPU 전체가 충분히 포화되어야만 이점을 내는데 이는 대형 행렬 곱셈에서만 성립함을 발견했다. 이는 모델 차원 4096 이상의 LLM에서만 일어난다.

순수 행렬 곱셈과 양자화 오버헤드에 대한 상세 벤치마크는 Table 5에 있다. cuBLASLt의 순수 Int8 행렬 곱셈은 모델 크기 5140(은닉 크기 20560)에서 cuBLAS보다 2배 빨라지기 시작한다. 입력을 양자화하고 출력을 역양자화해야 한다면(Transformer 전체를 Int8로 하지 않는 한 필수 요건), 16비트 대비 속도 향상은 모델 크기 5140에서 1.6배로 줄어든다. 모델 크기 2560 이하의 모델은 오히려 느려진다. 혼합 정밀도 분해를 더하면 추론이 더 느려져 13B와 175B 모델만 속도 향상을 갖는다.

이 수치들은 혼합 정밀도 분해용 CUDA 커널을 최적화하면 크게 개선될 수 있다. 다만 기존의 맞춤 CUDA 커널조차, 175B 모델을 제외한 모든 행렬 곱셈을 느리게 만드는 PyTorch·NVIDIA 기본 제공 양자화 커널보다는 훨씬 빠르다는 것도 확인할 수 있다.

GPT-3 Size	Small	Medium	Large	XL	2.7B	6.7B	13B	175B
Model dimension	768	1024	1536	2048	2560	4096	5140	12288
FP16-bit baseline	1.00x	1.00x	1.00x	1.00x	1.00x	1.00x	1.00x	1.00x
Int8 without overhead	0.99x	1.08x	1.43x	1.61x	1.63x	1.67x	2.13x	2.29x
Absmax PyTorch+NVIDIA	0.25x	0.24x	0.36x	0.45x	0.53x	0.70x	0.96x	1.50x
Vector-wise PyTorch+NVIDIA	0.21x	0.22x	0.33x	0.41x	0.50x	0.65x	0.91x	1.50x
Vector-wise	0.43x	0.49x	0.74x	0.91x	0.94x	1.18x	1.59x	2.00x
LLM.int8() (vector-wise+decomp)	0.14x	0.20x	0.36x	0.51x	0.64x	0.86x	1.22x	1.81x

Table 5: 크기가 다른 GPT-3 Transformer들의 피드포워드 첫 은닉 계층에 대한, 16비트 행렬 곱셈 대비 추론 속도 향상. 은닉 차원은 모델 차원의 4배다. "Int8 without overhead"의 속도 향상은 양자화·역양자화를 전혀 수행하지 않는다고 가정한 것이다. 1.0x보다 작은 수치는 속도 저하를 나타낸다. Int8 행렬 곱셈은 모델·은닉 차원이 큰 모델에서만 추론을 가속한다.

D.2 종단간 벤치마크 (End-to-end Benchmarks)

행렬 곱셈 벤치마크 외에, Hugging Face에서 BLOOM-176B의 종단간 추론 속도도 시험한다. Hugging Face는 어텐션 값을 캐싱하는 최적화된 구현을 사용한다. 이런 유형의 추론은 분산되어 있고 통신에 의존하므로, 전체 추론 런타임의 큰 부분이 고정 통신 오버헤드이기 때문에 Int8 추론으로 인한 전반적 속도 변화(향상/저하)는 더 작을 것으로 예상된다.

16비트와 비교하여 벤치마크하며, Int8 추론의 경우 더 큰 모델을 더 적은 장치에 넣을 수 있으므로 더 큰 배치 크기나 더 적은 GPU를 쓰는 설정도 시도한다. 결과는 Table 6에 있다. 전반적으로 Int8 추론은 약간 느리지만 16비트 추론과 토큰당 밀리초 지연이 거의 비슷하다.

Batch Size →	Hardware	1	8	32
bfloat16 baseline	8xA100 80GB	239	32	9.94
LLM.int8()	8xA100 80GB	253	34	10.44
LLM.int8()	4xA100 80GB	246	33	9.40
LLM.int8()	3xA100 80GB	247	33	9.11

Table 6: BLOOM-176B 모델의 여러 추론 유형 실행에 사용된 GPU 수에 대한 절제(ablation) 연구. 양자화된 BLOOM-176B와 원본 BLOOM-176B가 사용하는 GPU 수를 비교하고, 배치 크기별 토큰당 생성 속도(밀리초)를 보고한다. 다중 GPU 추론에는 HuggingFace의 accelerate 라이브러리로 구동되는 transformers 통합 버전을 사용한다. 우리의 방법은 원본 모델보다 더 적은 GPU에 들어가면서도 원본과 비슷한 성능에 도달한다.

부록 E. 학습 결과 (Training Results)

다양한 학습 설정에서 Int8 학습을 시험하고 32비트 기준선과 비교한다. 8비트 피드포워드 네트워크만 쓰는 설정, 여기에 어텐션 계층의 8비트 선형 투영을 더한 설정, 그리고 어텐션 자체까지 8비트로 하는 설정을 각각 시험하여 32비트 성능과 비교한다. 두 과제를 시험한다: (1) Books, CC-News, OpenWebText, CC-Stories를 포함한

RoBERTa 코퍼스 일부에 대한 언어 모델링, (2) WMT14+WMT16에 대한 신경망 기계 번역(neural machine translation, NMT).

결과는 Table 7과 Table 8에 있다. 학습의 경우, 어텐션의 선형 투영을 Int8 데이터 타입과 벡터 단위 양자화로 쓰면 NMT와 1.1B 언어 모델에서는 저하가 생기지만 209M 언어 모델링에서는 생기지 않음을 볼 수 있다. 혼합 정밀도 분해를 쓰면 결과가 약간 개선되지만 대부분의 경우 완전한 성능을 회복하기에는 부족하다. 이는 8비트 FFN 계층으로의 학습은 수월한 반면, 다른 계층들은 성능 저하 없는 대규모 8비트 학습을 위해 추가 기법이나 Int8이 아닌 다른 데이터 타입을 요구함을 시사한다.

Params	FFN 8-bit	Linear 8-bit	Attention 8-bit	Decomp	PPL
209M				0%	16.74
209M	X			0%	16.77
209M	X	X		0%	16.83
209M	X	X		2%	16.78
209M	X	X		5%	16.77
209M	X	X		10%	16.80
209M	X	X	X	2%	24.33
209M	X	X	X	5%	20.00
209M	X	X	X	10%	19.00
1.1B				0%	9.99
1.1B	X			0%	9.93
1.1B	X	X		0%	10.52
1.1B	X	X		1%	10.41

Table 7: 소규모·대규모 언어 모델링에 대한 초기 결과. 어텐션을 8비트로 수행하면 성능이 심각하게 저하되며 혼합 정밀도 분해로도 완전히 회복되지 않는다. 소규모 언어 모델은 8비트 FFN과 어텐션 계층의 8비트 선형 투영 모두에서 기준선에 가까운 성능을 보이지만, 대규모에서는 성능이 저하된다.

FFN 8-bit	Linear 8-bit	Decomp	BLEU
		0%	28.9
X		0%	28.8
X	X	0%	unstable
X	X	2%	28.0
X	X	5%	27.6
X	X	10%	27.5

Table 8: WMT14+16에 대한 8비트 FFN·선형 어텐션 계층의 신경망 기계 번역 결과. Decomp는 8비트 대신 16비트로 계산되는 비율을 나타낸다. BLEU 점수는 세 랜덤 시드의 중앙값이다.

부록 F. 미세조정 결과 (Fine-tuning Results)

GLUE에 대해 미세조정된 RoBERTa-large의 8비트 미세조정도 시험한다. 두 가지 설정을 실행한다: (1) 다른 Int8 방법들과의 비교, (2) 8비트 FFN 계층 및 8비트 어텐션 투영 계층에 대한 미세조정의 저하를 32비트와 비교. 5개 랜덤 시드로 미세조정하고 중앙값 성능을 보고한다.

Table 9는 미세조정을 위한 여러 기존 8비트 방법과 비교하며, 벡터 단위 양자화가 다른 방법들을 개선함을 보여준다. Table 10은 FFN 그리고/또는 선형 어텐션 투영을 8비트로 했을 때의 성능과, 혼합 정밀도 분해를 쓸 경우의 개선을 보여준다. 8비트 FFN 계층은 아무 저하도 일으키지 않는 반면, 8비트 어텐션 선형 투영은 크기 상위 2% 이상의 차원을 8비트 대신 16비트로 계산하는 혼합 정밀도 분해와 결합하지 않으면 저하를 일으킴을 발견했다. 이 결과는 성능을 저하시키지 않으려면 미세조정에서 혼합 정밀도 분해가 결정적 역할을 함을 부각한다.

Method	MNLI	QNLI	QQP	RTE	SST-2	MRPC	CoLA	STS-B	Mean
32-bit Baseline	90.4	94.9	92.2	84.5	96.4	90.1	67.4	93.0	88.61
32-bit Replication	90.3	94.8	92.3	85.4	96.6	90.4	68.8	92.0	88.83
Q-BERT	87.8	93.0	90.6	84.7	94.8	88.2	65.1	91.1	86.91
Q8BERT	85.6	93.0	90.1	84.8	94.7	89.7	65.0	91.1	86.75
PSQ	89.9	94.5	92.0	86.8	96.2	90.4	67.5	91.9	88.65
Vector-wise	90.2	94.7	92.3	85.4	96.4	91.0	68.6	91.9	88.81

Table 9: 피드포워드 계층을 8비트로, 나머지를 16비트로 하는 양자화 방법들의 GLUE 미세조정 결과. 혼합 정밀도 분해는 사용하지 않았다. 벡터 단위 양자화가 기준선들을 개선함을 볼 수 있다.

FFN 8-bit	Linear 8-bit	Decomp	MNLI	QNLI	QQP	RTE	SST-2	MRPC	CoLA	STS-B	MEAN
		0%	90.4	94.9	92.2	84.5	96.4	90.1	67.4	93.0	88.6
X		0%	90.2	94.7	92.3	85.4	96.4	91.0	68.6	91.9	88.8
X	X	0%	90.2	94.4	92.2	84.1	96.2	89.7	63.6	91.6	87.7
X	X	1%	90.0	94.6	92.2	83.0	96.2	89.7	65.8	91.8	87.9
X	X	2%	90.0	94.5	92.2	85.9	96.7	90.4	68.0	91.9	88.7
X	X	3%	90.0	94.6	92.2	86.3	96.4	90.2	68.3	91.8	88.7

Table 10: GLUE에 대한 8비트 피드포워드 네트워크(FFN)와 선형 어텐션 계층의 세부 결과. 점수는 5개 랜덤 시드의 중앙값이다. Decomp는 16비트 행렬 곱셈으로 분해되는 비율을 나타낸다. 추론과 비교하면, 선형 어텐션 계층까지 8비트로 변환하는 경우 미세조정은 더 높은 분해 비율을 필요로 하는 것으로 보인다.